3 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

3.1 Методика тестирования

Целью тестирования является подтверждение того, что программное средство выполняет все бизнес-операции в соответствии с требованиями, не допускает потери или двойного списания денег и устойчиво к типичным атакам на веб-приложения. Для банковской системы ошибка в расчёте или в проверке прав стоит дороже, чем ошибка в оформлении страницы, поэтому основное внимание уделено бизнес-логике, целостности данных и защите.

Тестирование выполнялось на нескольких уровнях, каждый из которых проверяет свою часть системы:

- модульные тесты проверяют отдельные функции без базы данных, такие как разбор и округление денежных сумм, расчёт контрольных цифр IBAN, проверку номера карты по алгоритму Луна, вычисление одноразовых кодов TOTP и построение графиков платежей;
- интеграционные тесты вызывают сервисы бизнес-логики с настоящей базой данных MySQL и проверяют проводки, остатки, статусы документов и записи журнала аудита;
- тесты программного интерфейса отправляют HTTP-запросы к приложению так же, как это делает браузер, с cookie сессии и токеном CSRF, и проверяют коды ответов, права ролей и сообщения об ошибках;
- сквозные проверки выполняются в браузере и повторяют действия пользователя в интерфейсе, от входа в систему до подтверждения операции кодом;
- функциональное тестирование по тестовым случаям подтверждает выполнение каждой бизнес-операции из спецификации требований.

Автоматические тесты написаны с использованием тестового фреймворка Vitest и библиотеки supertest, покрытие кода измеряется модулем @vitest/coverage-v8, а сквозные проверки выполняются библиотекой Playwright, которая управляет браузером Chromium. Все тесты запускаются одной командой npm test и не требуют ручной подготовки данных.

Для тестов используется отдельная база данных talerbank_test. Перед запуском она удаляется и создаётся заново из тех же SQL-скриптов, что и рабочая база, поэтому тесты проверяют настоящую схему со всеми ограничениями, внешними ключами и триггерами. Каждый тест сам создаёт нужных ему клиентов и счета и не зависит от результатов других тестов, а рабочая база данных при тестировании не затрагивается.

3.2 Автоматическое тестирование

Тесты сгруппированы по тем же предметным модулям, что и серверный код. Для каждого модуля в каталоге test есть папка с тестами сервисов и тестами программного интерфейса, общие функции создания клиентов, счетов

и HTTP-клиента с поддержкой сессии вынесены в отдельные вспомогательные файлы. Всего написано 294 теста в 29 файлах, объём тестового кода составляет около 4,5 тысячи строк при 6,8 тысячи строк серверного кода.

Тесты одного файла выполняются последовательно, а файлы запускаются по очереди, так как все они работают с одной тестовой базой данных. Полный прогон занимает около полутора минут, большая часть этого времени приходится на тесты программного интерфейса, которые выполняют вход пользователей и проводят операции через HTTP-запросы. Результат запуска автоматических тестов показан на рисунке 3.1.

```
> talerbank@0.1.0 test
> vitest run

RUN  v5.0.3 /home/user/js-course-work

✓ test/loans/api-loans.test.js (12 tests) 10551ms
✓ test/staff/api-staff.test.js (14 tests) 8825ms
✓ test/auth/security.test.js (15 tests) 8310ms
✓ test/transfers/api-transfers.test.js (14 tests) 8218ms
✓ test/admin/api-admin.test.js (26 tests) 7576ms
✓ test/admin/backup.test.js (3 tests) 5658ms
✓ test/accounts/api-accounts.test.js (11 tests) 5136ms
✓ test/deposits/api-deposits.test.js (10 tests) 4433ms
✓ test/app.test.js (16 tests) 4189ms
✓ test/payments/api-payments.test.js (5 tests) 4343ms
✓ test/auth/hardening.test.js (9 tests) 3414ms
✓ test/admin/admin.test.js (7 tests) 3658ms
✓ test/staff/operator.test.js (5 tests) 1824ms
✓ test/loans/loan.test.js (15 tests) 1215ms
✓ test/loans/scoring.test.js (8 tests) 480ms
✓ test/accounts/documents.test.js (3 tests) 922ms
✓ test/transfers/interbank.test.js (13 tests) 655ms
✓ test/transfers/races.test.js (10 tests) 622ms
✓ test/cards/card.test.js (7 tests) 615ms
✓ test/deposits/deposit.test.js (13 tests) 662ms
✓ test/transfers/transfer.test.js (15 tests) 552ms
✓ test/fraud/fraud.test.js (6 tests) 387ms
✓ test/payments/payment.test.js (10 tests) 272ms
✓ test/ledger/ledger.test.js (10 tests) 295ms
✓ test/notifications/mail.test.js (6 tests) 136ms
✓ test/admin/report.test.js (4 tests) 138ms
✓ test/shared/lib.test.js (11 tests) 11ms
✓ test/loans/schedule.test.js (11 tests) 8ms
✓ test/auth/totp.test.js (5 tests) 9ms

 Test Files 29 passed (29)
      Tests 294 passed (294)
   Start at 13:17:00
   Duration 97.73s (tests 89%, import 9%, transform 1%)
```

Рисунок 3.1 – Результат запуска автоматических тестов

Все 294 теста завершаются успешно. Тесты запускаются после каждого изменения кода, поэтому ошибка, внесённая при доработке одного модуля, сразу обнаруживается в тестах других модулей, которые от него зависят.

Отдельное внимание уделено проверке денежных инвариантов. После операций каждого вида тесты проверяют, что остаток каждого счёта равен разности оборотов по кредиту и дебету, сумма остатков всех счетов в каждой валюте равна нулю, а в оборотно-сальдовой ведомости обороты по дебету равны оборотам по кредиту. Тесты также подтверждают, что повторный запрос с тем же ключом идемпотентности не создаёт второй проводки, ошибка в середине документа откатывает все его проводки, а проводки и записи журнала аудита нельзя изменить или удалить даже прямым запросом к базе данных.

Тесты конкурентного доступа запускают несколько операций одновременно и проверяют, что блокировки строк в транзакциях работают правильно. Параллельные переводы с одного счёта не уводят его в минус, две заявки на перевод, созданные до подтверждения, вместе не превышают суточный лимит, из двух одновременных решений специалиста по одному подозрительному переводу срабатывает только одно, а десять одновременных попыток ввода неверного кода засчитываются как три.

Полнота тестирования оценивалась по покрытию кода, то есть по доле строк программы, которые выполнялись хотя бы в одном тесте. Покрытие строк вычисляется по формуле:

где – покрытие строк, %;

– число исполняемых строк, выполненных при тестировании;

– общее число исполняемых строк кода.

Покрытие ветвлений считается так же, но по числу направлений условных операторов, через которые прошло выполнение. Результаты измерения по модулям серверной части приведены в таблице 3.1.

Таблица 3.1 – Покрытие серверного кода автоматическими тестами

Модуль	Исполняемых строк	Покрытие строк, %	Покрытие ветвлений, %
Аутентификация и 2FA (auth)	265	99,2	90,5
Клиенты и анкеты (clients)	40	100,0	94,1
Счета и документы (accounts)	206	98,1	84,5
Двойная запись (ledger)	52	92,3	78,8
Переводы и клиринг (transfers)	285	98,9	86,7
Антифрод (fraud)	78	100,0	86,7

Модуль	Исполняемых строк	Покрытие строк, %	Покрытие ветвлений, %
Платежи (payments)	173	100,0	92,1
Карты (cards)	137	100,0	89,1
Вклады (deposits)	161	98,1	88,1
Кредиты и скоринг (loans)	460	98,9	86,5
Курсы валют (rates)	44	100,0	85,7
Настройки (settings)	56	100,0	88,0
Рабочее место операциониста (staff)	49	100,0	92,9
Администрирование (admin)	246	95,9	81,8
Уведомления и почта (notifications)	64	90,6	77,8
Главные страницы (dashboard)	98	77,6	66,0
Журнал аудита (audit)	5	40,0	93,3
Общие функции (shared)	181	95,6	83,4
Ядро приложения (app, config, db/pool)	69	95,7	82,1
Скрипты запуска и заполнения базы	63	0,0	0,0
Итого	2732	95,0	84,9

Модули с денежными операциями покрыты практически полностью, непокрытыми в них остались в основном ветви обработки неожиданных ошибок базы данных. Наименьшее покрытие у модуля главных страниц, где часть ветвей относится к показателям ролей, для которых в тестовых данных нет записей. Скрипты запуска сервера, применения миграций и заполнения демонстрационных данных выполняются отдельными командами, а не в тестах, и проверяются при каждом развёртывании приложения.

3.3 Функциональное тестирование

Функциональное тестирование подтверждает, что каждая бизнес-операция из спецификации функциональных требований выполняется от начала до конца и правильно обрабатывает ошибки пользователя. Для каждой операции составлен тестовый случай с исходными данными и ожидаемым результатом. Каждый случай проверяется автоматическими тестами программного интерфейса, а операции перевода, входа и проверки прав дополнительно выполнены в интерфейсе на демонстрационных данных. Тестовые случаи и результаты их выполнения приведены в таблице 3.2.

Таблица 3.2 – Тестовые случаи функционального тестирования

№	Действие и исходные данные	Ожидаемый результат	Результат
1	Регистрация клиента с корректной анкетой, проверка анкеты операционистом	Анкета подтверждена, клиенту открыт текущий счёт в BYN	Пройден
2	Открытие счёта в долларах США	Счёт открыт, номер IBAN проходит проверку контрольных цифр	Пройден
3	Перевод со своего счёта в USD на свой счёт в BYN	Зачислена сумма по курсу банка, двойная запись сходится	Пройден
4	Перевод 150 BYN клиенту банка, ввод неверного, затем верного кода	Сообщение «Неверный код. Осталось попыток: 2», после верного кода перевод проведён	Пройден
5	Перевод на IBAN с неверными контрольными цифрами	Ошибка у поля номера счёта, перевод не создан	Пройден
6	Перевод 3000 BYN при остатке 2000 BYN	Сообщение «Недостаточно средств на счёте»	Пройден
7	Перевод суммы больше суточного лимита	Сообщение «Превышен суточный лимит переводов»	Пройден
8	Крупный перевод новому получателю	Перевод задержан антифродом, деньги зарезервированы до решения специалиста	Пройден
9	Оплата услуги по каталогу с сохранением шаблона	Деньги списаны, сформирована квитанция PDF, шаблон заполняет форму	Пройден
10	Открытие вклада и досрочное закрытие	Вклад возвращён с процентами по ставке до востребования	Пройден
11	Заявка на кредит, рассмотрение специалистом, подписание кодом	Кредит выдан на счёт, построен график платежей	Пройден
12	Частичное досрочное погашение кредита	Остаток долга уменьшен, график пересчитан	Пройден
13	Выпуск карты, установка лимита, блокировка	Покупка по заблокированной карте отклоняется	Пройден

№	Действие и исходные данные	Ожидаемый результат	Результат
14	Выписка по счёту за период	Входящий остаток, операции и исходящий остаток согласованы, файл PDF открывается	Пройден
15	Закрытие операционного дня	Банковская дата сдвинута, начислены проценты и списаны плановые платежи	Пройден
16	Блокировка пользователя администратором	Заблокированный пользователь не может войти, событие записано в журнал	Пройден

Особое внимание при функциональном тестировании уделялось сообщениям об ошибках. Каждая ошибка ввода должна выводиться рядом с полем, к которому она относится, и объяснять, что нужно исправить, а уже введённые данные не должны теряться.

При вводе номера счёта с неверными контрольными цифрами сервер отклоняет запрос с кодом 422 и указывает поле с ошибкой, а интерфейс выделяет это поле и выводит под ним текст ошибки. Остальные поля формы сохраняют введённые значения. Сообщение о неверном номере счёта показано на рисунке 3.2.

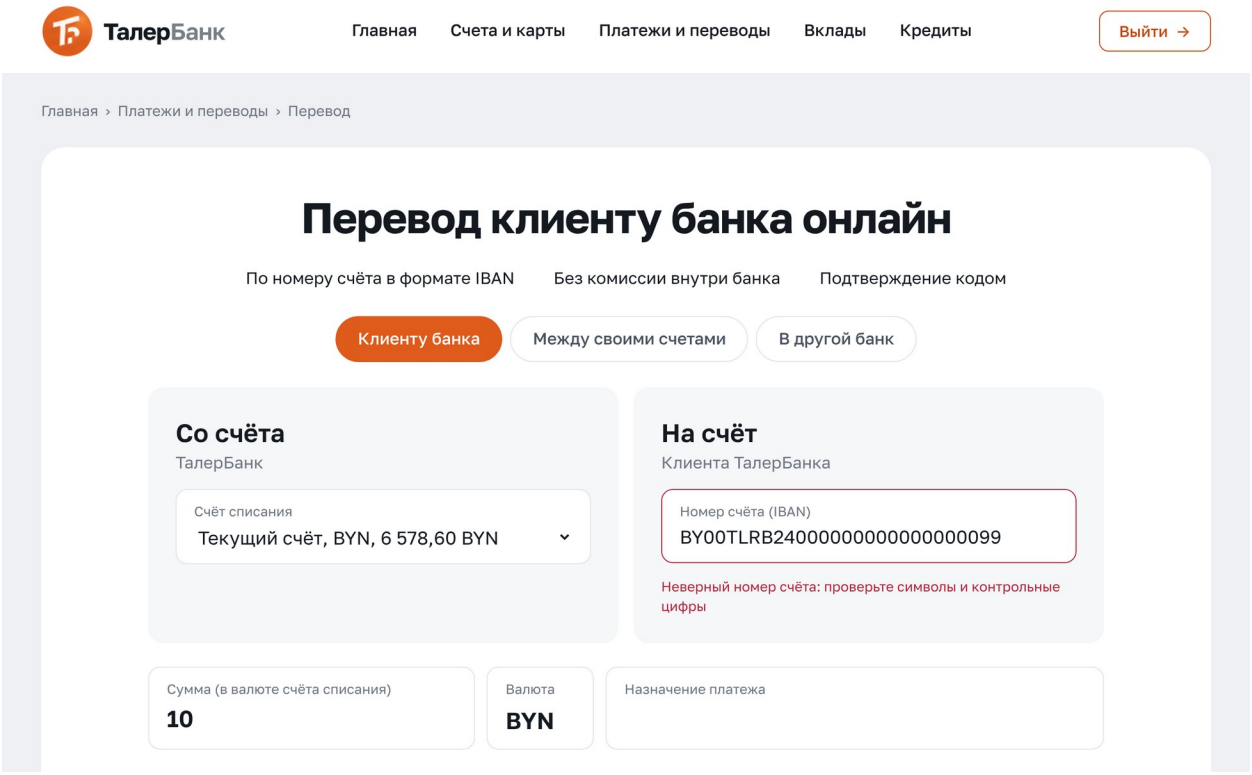


Рисунок 3.2 – Проверка номера счёта в форме перевода

При вводе неверного кода подтверждения перевод не проводится, а пользователь видит число оставшихся попыток. После трёх неверных попыток код аннулируется и перевод нужно оформить заново, что не позволяет подобрать код перебором. Сообщение о неверном коде подтверждения показано на рисунке 3.3.

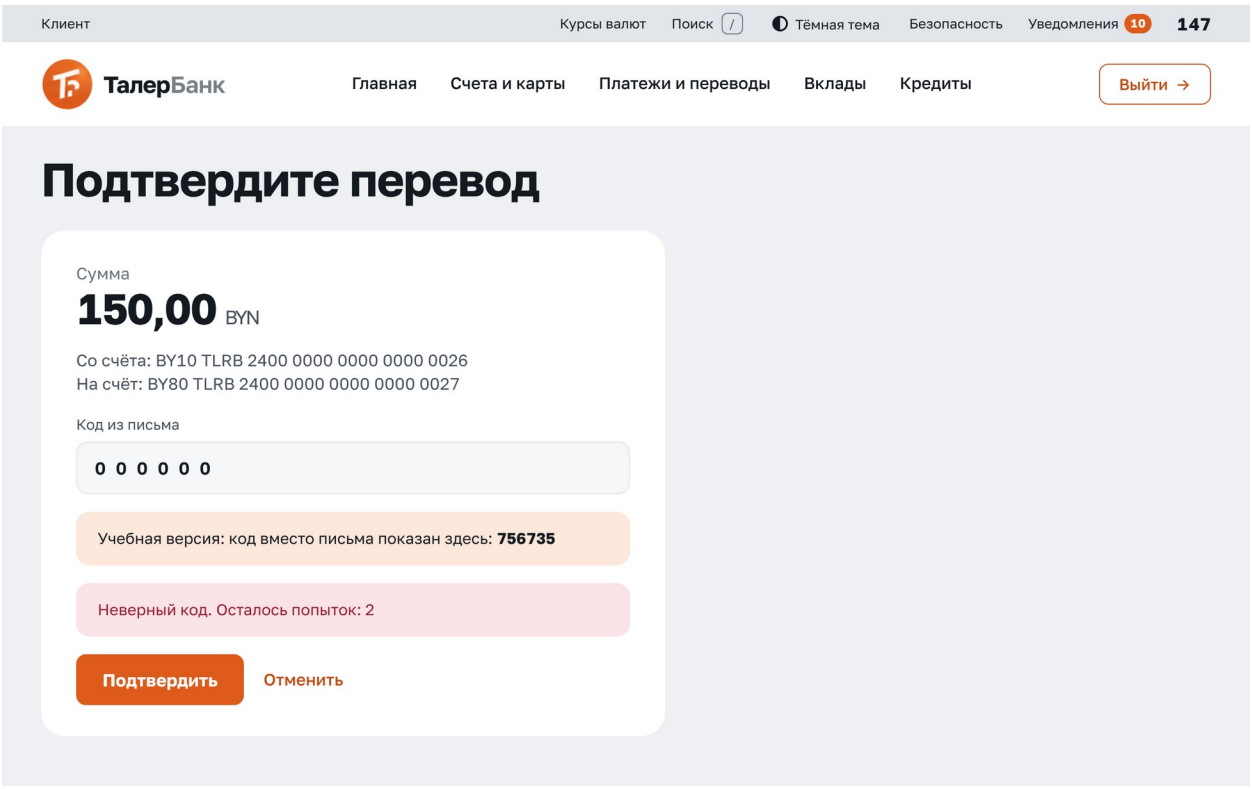


Рисунок 3.3 – Неверный код подтверждения перевода

3.4 Проверка защищённости

Механизмы защиты проверялись попытками выполнить типичные атаки на веб-приложения. Часть проверок входит в автоматические тесты и повторяется при каждом запуске, остальные выполнены в браузере сценариями Playwright.

Попытка входа с логином и паролем вида «anna' OR '1'='1» завершается отказом, так как все запросы к базе данных выполняются с параметрами и введённая строка не становится частью SQL-запроса. Поиск по журналу и спискам ищет символы % и _ буквально, поэтому пользователь не может через строку поиска получить лишние записи.

Комментарий к переводу с текстом <script>alert(1)</script> сохраняется и выводится в истории операций как обычный текст. Вся разметка в интерфейсе строится функцией, которая экранирует подставляемые значения, а политика безопасности содержимого запрещает выполнение встроенных скриптов, даже если бы экранирование было пропущено.

Изменяющий запрос без токена CSRF или с неверным токеном отклоняется с кодом 403, перевод при этом не создаётся. Токен с недопустимыми символами также даёт отказ 403, а не внутреннюю ошибку сервера. После входа выдаются новые идентификатор сессии и токен, поэтому токен, полученный до входа, использовать нельзя.

Тесты разграничения прав обращаются к каждому маршруту от имени гостя, клиента и сотрудников разных ролей. Гость получает ответ 401, пользователь без нужного права получает 403, а клиент не может открыть, подтвердить или отменить чужой перевод, счёт, карту, вклад или кредит, даже если подставит в адрес их номер. При смене роли права сотрудника меняются сразу, без повторного входа.

Проверка подбора пароля показала, что после пяти неверных попыток учётная запись блокируется, а чередование верного пароля с неверными кодами второго фактора также приводит к блокировке. Тесты двухфакторной защиты сверяют вычисленные коды с контрольными значениями из RFC 4226 и RFC 6238 и проверяют, что резервный код и код одного интервала принимаются только один раз.

Тесты шифрования подтверждают, что одни и те же паспортные данные при каждом шифровании дают разный шифртекст, а изменение хотя бы одного байта шифртекста обнаруживается при расшифровке. Резервная копия восстанавливается в исходное состояние, повреждённый файл копии и подставленное имя файла отклоняются.

3.5 Проверка в браузерах

Требования задания предусматривают работу приложения в браузерах Яндекс Браузер, Firefox, Chrome и Opera в среде Windows 10 и новее. Интерфейс использует только стандартные возможности HTML5, CSS3 и JavaScript, которые поддерживаются всеми этими браузерами, а возможности, появившиеся недавно, такие как плавные переходы между страницами, применяются только там, где браузер их поддерживает, и не влияют на работу при их отсутствии.

Сквозные сценарии выполнены в браузере Chromium, на котором основаны Chrome, Яндекс Браузер и Opera, при ширине окна 1280 пикселей и на ширине экрана телефона 390 пикселей. Ошибок выполнения JavaScript в консоли не возникало, страницы не выходили за ширину экрана.

[Дополнить после проверки в Яндекс Браузере, Firefox и Opera на Windows: версии браузеров и рисунок со страницей в каждом из них.]

3.6 Анализ результатов тестирования

В ходе тестирования был выявлен ряд дефектов, большая часть которых проявлялась только при одновременных запросах или при нестандартных входных данных и не была заметна при обычной работе с интерфейсом. Все найденные дефекты исправлены, и для каждого из них добавлен тест, который

воспроизводит исходную ситуацию. Основные дефекты и способы их исправления приведены в таблице 3.3.

Таблица 3.3 – Дефекты, выявленные при тестировании

Дефект	Как обнаружен	Исправление
Две заявки на перевод, созданные до подтверждения, вместе превышали суточный лимит	Тест конкурентного доступа	Лимит проверяется повторно при подтверждении под блокировкой строки пользователя
Одновременные неверные попытки ввода кода засчитывались не все, и попыток было больше трёх	Тест с десятью параллельными запросами	Проверка кода выполняется в транзакции с блокировкой строки кода
Одновременные «одобрить» и «отклонить» по одному случаю антифрода срабатывали оба	Тест конкурентного доступа	Решение меняет статус условным обновлением, второе решение отклоняется
Отмена перевода после его проведения меняла статус «проведён» на «отменён»	Тест смены статусов	Статус меняется только из допустимого предыдущего статуса
Учётная запись блокировалась на одну неверную попытку раньше заданного числа	Тест подбора пароля	Изменён порядок присваиваний в запросе, так как MySQL вычисляет их слева направо
Нечисловой идентификатор в адресе и токен CSRF с не-ASCII символами вызывали ошибку 500	Тесты с некорректными данными	Идентификаторы проверяются до обращения к базе, токен сравнивается побайтово

По результатам тестирования все 294 автоматических теста и все 16 тестовых случаев функционального тестирования выполняются успешно. Покрытие серверного кода тестами составляет 95,0 % по строкам и 84,9 % по ветвлениям, а модули, отвечающие за деньги, покрыты почти полностью. Проверка защищённости не выявила возможности внедрения SQL-кода и сценариев, подделки межсайтовых запросов, доступа к чужим данным и подбора паролей и кодов.

Таким образом, программное средство выполняет все бизнес-операции, указанные в требованиях, сохраняет целостность денежных данных при одновременной работе пользователей и защищено от типичных атак на веб-приложения. Набор автоматических тестов позволяет дорабатывать систему без риска незаметно нарушить уже работающие функции.